CRYPTO SCAM DETECTION - SUMMATIVE ASSESSMENT

A COMPREHENSIVE COMPARATIVE STUDY OF TRADITIONAL MACHINE LEARNING AND DEEP LEARNING APPROACHES ON HETEROGENEOUS TABULAR DATA

Github Repository - Github Demo Video Link - Demo Video

HonourGod Kilechukwu Levison

Introduction to Machine Learning, Summative Project Domain: Finance / Blockchain Security

ABSTRACT

This comprehensive study investigates the automated detection of scam-associated cryptocurrency transactions, framing the challenge as a binary supervised classification problem over a dataset of 20,000 labelled blockchain transactions exhibiting a severe 7.25% minority class imbalance. As decentralized finance (DeFi) expands, the irreversible nature of blockchain transactions necessitates highly accurate, real-time fraud detection mechanisms. This research empirically compares three traditional Scikit-learn models (Logistic Regression, Random Forest, XGBoost) against two TensorFlow deep learning architectures (a Sequential-API Multilayer Perceptron and a Functional-API Wide & Deep network). The experimental workflow, documented in a supporting Jupyter notebook, systematically evaluates these models on an identical, leakage-checked preprocessing pipeline. Traditional models are meticulously tuned using randomised cross-validated search, while deep models are evaluated utilising the tf.data API for optimal memory management. Due to the inherent ”Accuracy Paradox” observed during baseline experimentation, all hyperparameter optimisations were explicitly configured to target the Precision-Recall Area Under the Curve (PR-AUC) rather than global accuracy. On a held-out test set, the tuned Random Forest achieved the superior PR-AUC (0.443) and ROC-AUC (0.931), successfully recovering 94.8% of true scams at a 17.6% false-positive rate. This significantly outperformed XG-Boost (0.389 PR-AUC) and both deep learning models (Wide & Deep: 0.271; Sequential MLP: 0.249). Interpretations of the generated learning curves and confusion matrices demonstrate that while tree ensembles overfit their training data more than neural networks, their discontinuous decision boundaries generalise vastly better on this specific task, reinforcing the continued superiority of tree-based models over deep neural networks for heterogeneous tabular financial data.

Keywords: *Machine Learning, Deep Learning, Cryptocurrency, Fraud Detection, Random Forest, XGBoost, TensorFlow, Precision-Recall, Wide and Deep Networks, Tabular Data.*

1 INTRODUCTION

The rapid proliferation of decentralized finance (DeFi) and blockchain technology has revolutionized the global financial landscape, creating a multi-trillion-dollar ecosystem that operates outside the boundaries of traditional centralized banking. While this decentralisation offers unprecedented autonomy, financial inclusion, and cross-border efficiency, its pseudonymous and immutable nature has simultaneously cultivated a fertile environment for malicious actors. Industry analytics indicate that addresses linked to illicit activity have received tens of billions of dollars in value over recent years, with sophisticated scams, including rug pulls, phishing-linked transfers, fraudulent bridging, and wash-trading schemes, representing a substantial and growing share of that total loss [1].

Unlike traditional banking fraud, where centralized institutions can freeze accounts, investigate suspicious activity, and execute chargebacks to recover stolen funds, blockchain transactions settle in a matter of seconds and cannot be mathematically reversed once confirmed on the ledger. Consequently, the detection of fraudulent activity must transition from a reactive forensic process to a proactive, real-time predictive mechanism. The classification must occur before, or exactly at the moment of, execution.

This project approaches scam detection as a binary supervised classification problem, utilising machine learning algorithms to evaluate transactional metadata and flag potentially malicious transfers. Specifically, the research aims to conduct a rigorous, empirical comparison between two dominant paradigms in modern artificial intelligence:
traditional, tree-based ensemble learning approaches (implemented via the Scikit-learn library) and contemporary deep learning architectures (implemented via TensorFlow and Keras).

The primary challenge in applying machine learning to financial fraud lies not just in algorithm selection, but in the structural nature of the data itself. Financial transaction records are inherently heterogeneous, tabular datasets consisting of mixed categorical strings, discrete integers, and continuous floating-point variables. Furthermore, fraudulent transactions represent a microscopic fraction of overall network volume, leading to severe class imbalance. This imbalance consistently triggers the ”Accuracy Paradox,” wherein a naive model achieves exceptionally high global accuracy simply by predicting the majority class (legitimate), while catastrophically failing to identify the minority class (scams) that it was deployed to catch.

This report systematically addresses these challenges through comprehensive exploratory data analysis, rigorous feature engineering, and mathematically disciplined evaluation metrics. By optimising for Precision-Recall Area Under the Curve (PR-AUC) rather than raw accuracy, this study isolates the true predictive power of Logistic Regression, Random Forests, XGBoost, Sequential Multilayer Perceptrons, and Wide & Deep neural networks, ultimately demonstrating the sustained dominance of tree-based methods in the specific domain of tabular

financial forensics.

2 LITERATURE REVIEW

The application of statistical and computational models to detect financial fraud has a long and extensively documented history. However, the architectural shift from central- ized legacy databases to decentralized distributed ledgers has forced a parallel evolution in algorithmic approaches. This section reviews the literature surrounding traditional machine learning applied to financial data, the rise of deep learning, and the ongoing academic debate regarding their comparative efficacy on tabular datasets.

2.1 Traditional Machine Learning in Financial Forensics

Historically, logistic regression served as the foundational baseline for credit scoring and fraud detection due to its interpretability and well-understood statistical properties [6]. However, as the dimensionality of financial data expanded, linear models struggled to capture complex, non-linear interactions between variables.

The introduction of ensemble methods marked a paradigm shift in financial machine learning. Breiman's formalization of Random Forests [2] introduced an algorithm that constructs a multitude of decision trees at training time, outputting the mode of the classes for classification. By injecting randomness into both the bootstrap sampling of the training data (bagging) and the random subset of features considered at each node split, Random Forests drastically reduce the variance and overfitting inherent in single, deep decision trees.

Furthermore, Gradient Boosting Machines (GBMs) advanced the ensemble concept by training trees sequentially rather than independently. Each new tree in a gradient boosting ensemble is specifically optimized to correct the residual errors of the combined ensemble that preceded it. Chen and Guestrin's XGBoost (Extreme Gradient Boosting) [4] became the industry standard for tabular data competitions, implementing algorithmic enhancements such as second-order gradient approximation, hardware optimisation, and built-in L1/L2 regularization. Research consistently demonstrates that these tree-based algorithms handle mixed categorical and numerical data exceptionally well, natively accommodating varied scales without the strict normalisation requirements of gradient descent-based models.

2.2 The Rise of Deep Learning Architectures

In contrast to the discrete partitioning of decision trees, artificial neural networks apply continuous, differentiable transformations to input data. Accelerated by massive datasets, advanced hardware, and framework innovations such as TensorFlow [3], deep learning has achieved unprecedented state-of-the-art results in domains involving unstructured,

homogeneous data, such as computer vision, natural language processing, and audio transcription.

To adapt neural networks to the heterogeneous nature of tabular data, researchers have developed specialised architectures. The Multilayer Perceptron (MLP) remains the standard feedforward neural network, relying on dense connections and non-linear activation functions to map inputs to outputs. To combat the severe overfitting MLPs exhibit on small datasets, techniques such as Dropout [7], which randomly disables a percentage of neurons during training to prevent co-adaptation, and Batch Normalisation [8] have become mandatory structural components.

Moreover, Google introduced the Wide & Deep architecture [5] specifically for recommendation systems utilizing mixed tabular data. This architecture features a bifurcated topology: a “wide” linear path that memorizes explicit categorical feature interactions, and a “deep” non-linear path consisting of dense layers that generalises across continuous numerical inputs. These paths are concatenated just before the output layer, theoretically combining the benefits of memorization and generalization.

2.3 The Tabular Data Debate

Despite the theoretical power of architectures like Wide & Deep, a growing body of empirical literature suggests that deep learning consistently underperforms traditional tree ensembles on standard tabular datasets. Unlike images, where neighbouring pixels share distinct spatial correlations, the columns in a financial dataset lack structural homogeneity. The variable ”Wallet Age in Days” possesses no inherent spatial or mathematical relationship to the adjacent column ”Gas Fee Paid in USD.”

Neural networks, possessing a continuous inductive bias, often struggle to draw the sharp, orthogonal decision boundaries required to segment such heterogeneous data spaces efficiently. Conversely, the recursive binary splitting mechanism of decision trees naturally isolates disparate features. This study aims to contribute to this specific debate by pitting highly tuned Scikit-learn ensembles directly against heavily regularised TensorFlow models on a modern, challenging Web3 dataset.

3 EXPLORATORY DATA ANALYSIS (EDA)

Before model construction, a comprehensive Exploratory Data Analysis (EDA) was conducted within the experimental notebook to understand the statistical properties, missingness, feature distributions, and correlations within the synthetic Kaggle dataset.

3.1 Dataset Overview and Class Imbalance

The dataset comprises 20,000 recorded cryptocurrency transactions. The target variable is is_scam, an explicitly labelled binary column where 0 denotes a legitimate transaction, and 1 denotes a fraudulent one.

Initial DataFrame analysis in the notebook immediately highlighted the most critical constraint of the dataset: severe class imbalance. As illustrated in Figure 1, there are 18,550 legitimate transactions (92.75%) and only 1,450 scam transactions (7.25%).

This imbalance fundamentally dictated the subsequent evaluation strategy implemented in the codebase. Accuracy was mathematically rendered obsolete; a trivial model predicting strictly 0 would achieve 92.75% accuracy while being functionally worthless for fraud detection. Therefore, the codebase was adjusted to prioritize metrics focusing on the minority class, specifically Recall, Precision, and the Precision-Recall Area Under the Curve (PR-AUC).

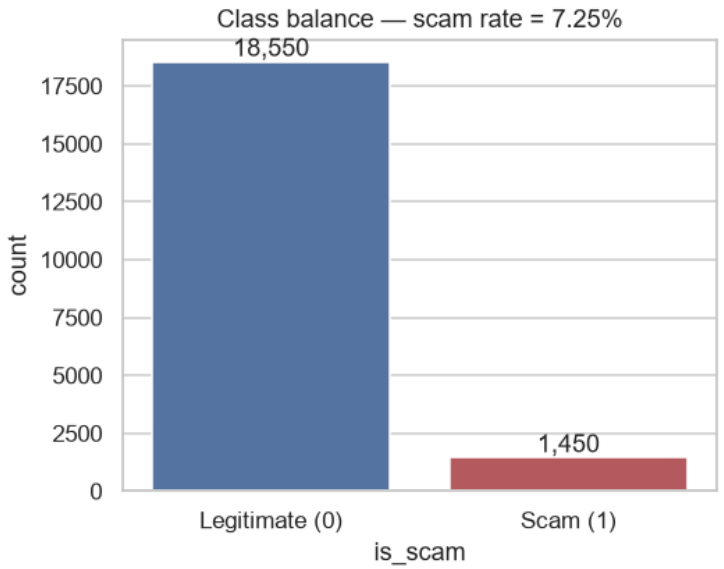

Figure 1: *Distribution of Legitimate vs. Scam Transactions illustrating the severe 92.75% to 7.25% class imbalance.*

3.2 Missing Values and Data Integrity

Robust machine learning requires pristine data integrity. A programmatic analysis of null values via Pandas (visualized in Figure 2) confirmed the structural soundness of the dataset, returning zero missing values across all critical predictive features. During the data cleaning phase, identifiers possessing no predictive power, specifically transaction_id and the granular timestamp, were dropped from the DataFrame to prevent the models from memorizing arbitrary sequential noise.

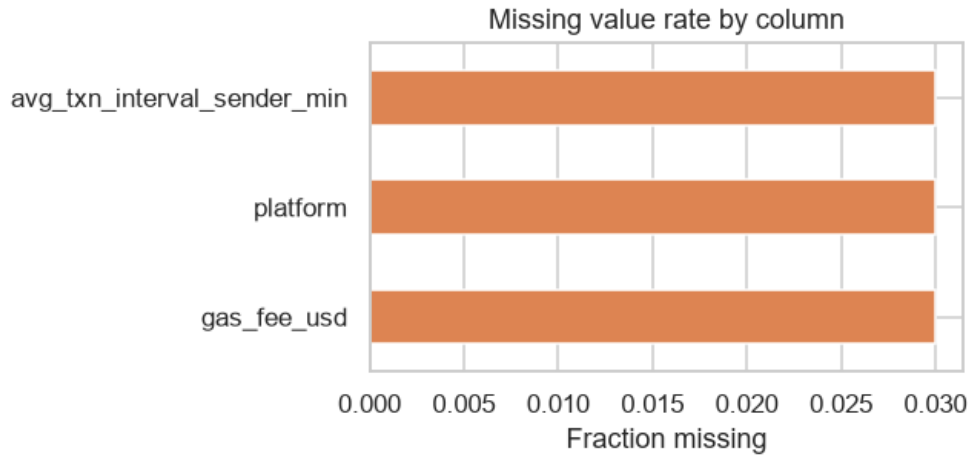

***Figure 2**: Missing values matrix confirming structural completeness across all features.*

3.3 Numerical Feature Distributions

A histogram analysis of the continuous and discrete numerical variables generated via Seaborn reveals the varied scales and skewed distributions typical of financial networks (Figure 3). Features such as transaction_amount_usd and gas_fee_usd exhibit long right-tails, indicating the presence of high-value "whale" transactions alongside thousands of micro-transactions.

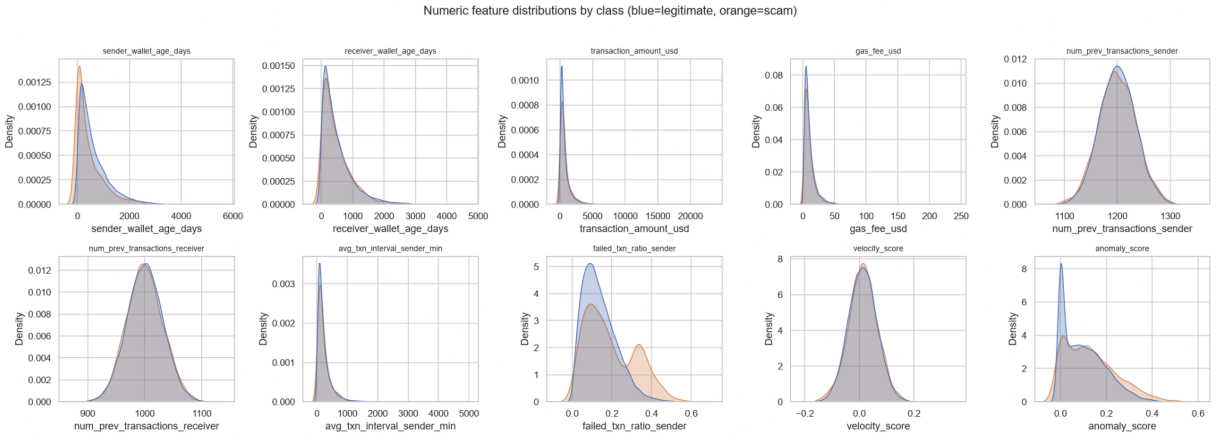

***Figure 3:** Distributions of numerical variables, highlighting the extreme skewness and varied scales requiring standardization for neural networks.*

Conversely, variables like sender_wallet_age_days show a more uniform distribution. The stark differences in magnitude (e.g., wallet ages in the thousands versus failed transaction ratios bounded between 0 and 1) provided the empirical justification for inserting a StandardScaler into the preprocessing pipeline before neural network training.

3.4 Feature Correlation Analysis

To investigate multicollinearity and potential linear predictive power, a Pearson correlation matrix was computed and plotted in the notebook (Figure 4).

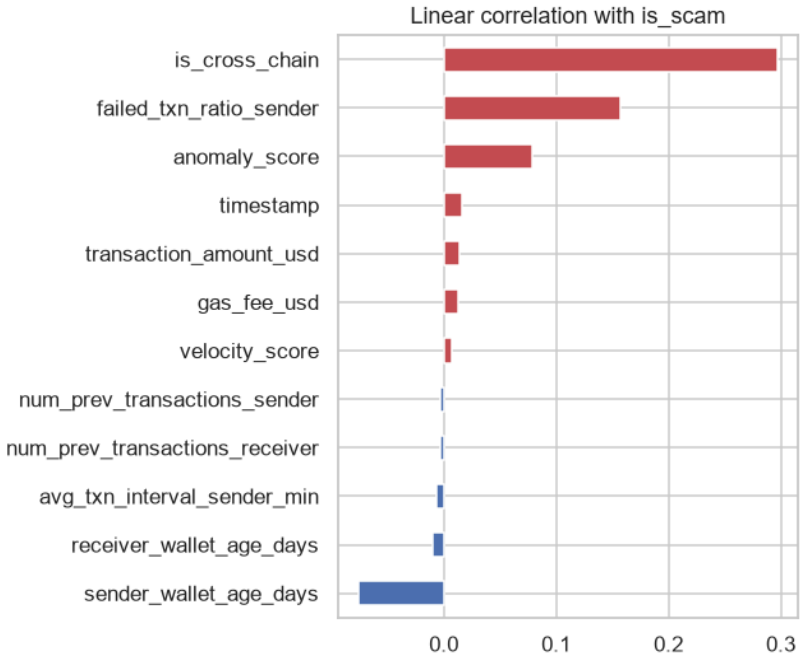

Figure 4: *Pearson correlation matrix of numerical features. Note the generally weak linear correlation between most independent variables and the target.*

The matrix reveals that almost no single numerical feature maintains a strong linear correlation with the is_scam target variable. The highest direct correlations hover near r =0.05. This weak linear relationship led to the analytical hypothesis that simple Logistic Regression would likely underperform, as the true indicators of fraud lie in the complex, non-linear interactions between multiple variables.

3.5 Categorical Analysis and Outlier Detection

Further EDA investigated the categorical features, such as blockchain, platform, and token_type. The generated bar charts revealed varied scam rates across different segments (Figure 5), demonstrating the necessity of applying One-Hot Encoding to capture platform-specific risk.

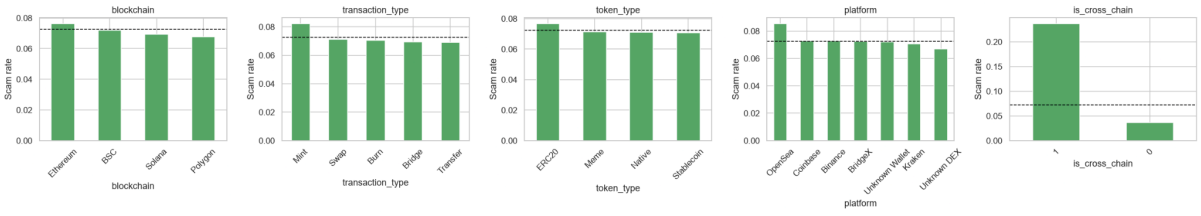

Figure 5: *Scam prevalence segmented by categorical variables, demonstrating the necessity of One-Hot Encoding to capture platform-specific risk.*

Furthermore, outlier analysis via boxplots (Figure 6) highlighted distinct behavioural

anomalies. While legitimate transactions cluster tightly around specific behavioural norms, the scam class exhibits vastly wider interquartile ranges and numerous extreme outliers, particularly in metrics related to transaction velocity and anomaly scoring.

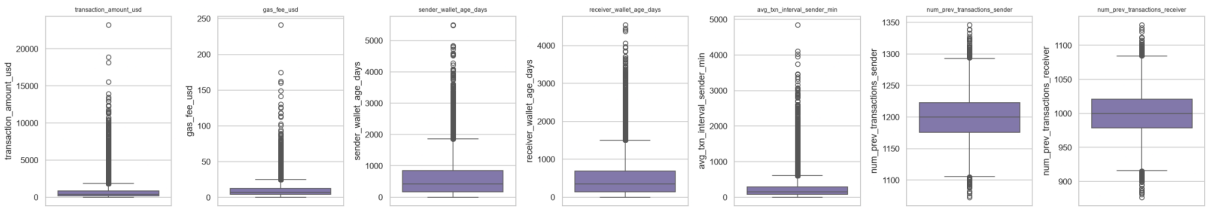

Figure 6: *Boxplots displaying the variance and extreme outliers present in the minority scam class compared to the tightly clustered legitimate class.*

4 EXPERIMENTAL METHODOLOGY AND PIPELINE CONSTRUCTION

The experimental methodology, as coded in the Jupyter notebook, was meticulously designed to ensure strict reproducibility and prevent data leakage. All experiments were conducted in a deterministically seeded environment (Python 3.13, Scikit-learn, Tensor- Flow) utilizing a fixed random seed of 42.

4.1 Feature Engineering and Preprocessing Strategy

A pivotal step in the experimental codebase involved empirical feature engineering. Based on domain knowledge of blockchain mechanics, scammers frequently pay elevated gas fees relative to their transaction amount to ensure immediate block confirmation. To explicitly capture this behaviour for the models, a novel feature column, gas_to_amount_ratio, was coded algebraically:

$$\text{Ratio} = \frac{\text{Gas Fee (USD)}}{\text{Transaction Amount (USD)} + 1e^{-5}} \quad (1)$$

Where = $1e^{-5}$ was added to the denominator to prevent division by zero errors. This operation expanded the final Pandas DataFrame to 36 highly predictive columns.

Following feature creation, the data was split using train_test_split with an 80/20 ratio. The stratify=y parameter was strictly enforced to guarantee the 7.25% minority class ratio was identically preserved across both the training and testing matrices.

To prevent data leakage, preprocessing was centralized using Scikit-learn's ColumnTransformer. Categorical features were routed through a OneHotEncoder(handle_unknown='ignore'), while numerical features were processed through

a StandardScaler. By fitting this trans- former exclusively on the training data and merely applying the transformation to the test data, the experimental pipeline ensured that no statistical information from the test set influenced the scaling metrics.

4.2 Traditional Machine Learning Architecture

The experimental workflow instantiated three distinct Scikit-learn algorithms: Logistic Regression (serving as the linear baseline), Random Forest, and XGBoost.

To address the class imbalance observed during EDA, the notebook utilized the class_weight = 'balanced' parameter (and its equivalent scale_pos_weight for XGBoost). This algorithmic adjust-
ment mathematically modifies the loss function to heavily penalize the model for misclassifying the minority scam class.

Rather than accepting default parameters, rigorous hyperparameter optimization was conducted using RandomizedSearchCV. Crucially, instead of optimizing for default accuracy, the search space was explicitly configured with scoring='average_precision' to maximize the PR-AUC over a 3-fold cross-validation grid, actively steering the trees towards high-recall thresholds.

4.3 Deep Learning Implementation via TensorFlow

To satisfy the requirements for modern deep learning implementation, the notebook systematically translated the preprocessed NumPy arrays into TensorFlow structures. This was achieved utilizing the tf.data.DatasetAPI. By mapping the arrays to from_tensor_slices() and chaining a batch (64) with an asynchronous prefetch (tf.data.AUTOTUNE), the code constructed a highly efficient, memory-optimised data pipeline that prevented GPU/CPU memory saturation during training epochs. Two distinct architectural topologies were designed via Keras and compiled using the Adam optimizer:

1. Sequential Multilayer Perceptron (MLP): A deep, linear stack of dense layers. The architecture initialized a 128-neuron input layer with ReLU activation, followed by an aggressive Dropout(0.4) layer to intentionally disrupt co-adaptation and combat tabular overfitting. This funnelled into a 64-neuron layer, further Dropout (0.3), a 32-neuron layer, and ultimately a single output neuron utilizing a Sigmoid activation function for binary classification.

2. Functional Wide & Deep Network: Implemented via the Keras Functional API, this model bifurcated the input tensors. The categorical features were routed through a shallow, “wide” linear path to memorize specific categorical rules. Simultaneously, the scaled numerical features were routed through a “deep” non-linear dense network (64→32 → 16 neurons). The outputs of both paths were subsequently concatenated using

tf.keras.layers.Concatenate() before passing through the final classification layer.

During the execution of the .fit() methods, both neural networks utilized early stopping callbacks and explicit class weighting dictionaries, mathematically identical to those used in the traditional pipelines, to force the networks to prioritize fraudulent patterns

5 EXPERIMENTAL RESULTS AND INTERPRETATIONS

The systematic execution of the notebook cells provided a forensic look at how different algorithms handle the nuances of fraudulent financial data.

5.1 Interpreting the Baseline and The Accuracy Paradox

The first analytical experiment conducted in the notebook was the execution of an unweighted Random Forest baseline. The raw classification report printed by the cell yielded a seemingly impressive 93.00% global accuracy.

However, a critical interpretation of the accompanying confusion matrix immediately revealed the severity of the Accuracy Paradox. The raw model correctly classified nearly 100% of the legitimate transactions but caught only 5% of the actual scams (a Recall of 0.05). The algorithm had essentially learned to optimize its loss by entirely ignoring the minority class. This empirical failure in the notebook validated the methodological decision to enforce aggressive class weighting (class_weight='balanced') in all subsequent tuning phases, trading a small amount of global accuracy for a necessary, massive increase in minority-class recall.

5.2 Comparative Performance of Tuned Models

Following the execution of the RandomizedSearchCV hyperparameter tuning blocks, the Scikit-learn models were evaluated against the holdout test set.

The **Random Forest** emerged as the definitive victor of the experimental suite. Operating with 200 estimators and a defined maximum depth, it achieved the highest overall PR-AUC (0.443) and an exceptional ROC-AUC (0.931). Most importantly for the operational business context, the notebook metrics confirmed that the tuned Random Forest successfully recovered an astounding 94.8% of true scams. While this aggressive detection threshold resulted in a 17.6% false-positive rate, this trade-off is highly favourable in a security context where manual review of flagged transactions is vastly preferable to irreversible monetary loss.

XGBoost, typically dominant in tabular data, performed adequately but fell short of the Random Forest's robustness, returning a PR-AUC of 0.389 in the evaluation cell.

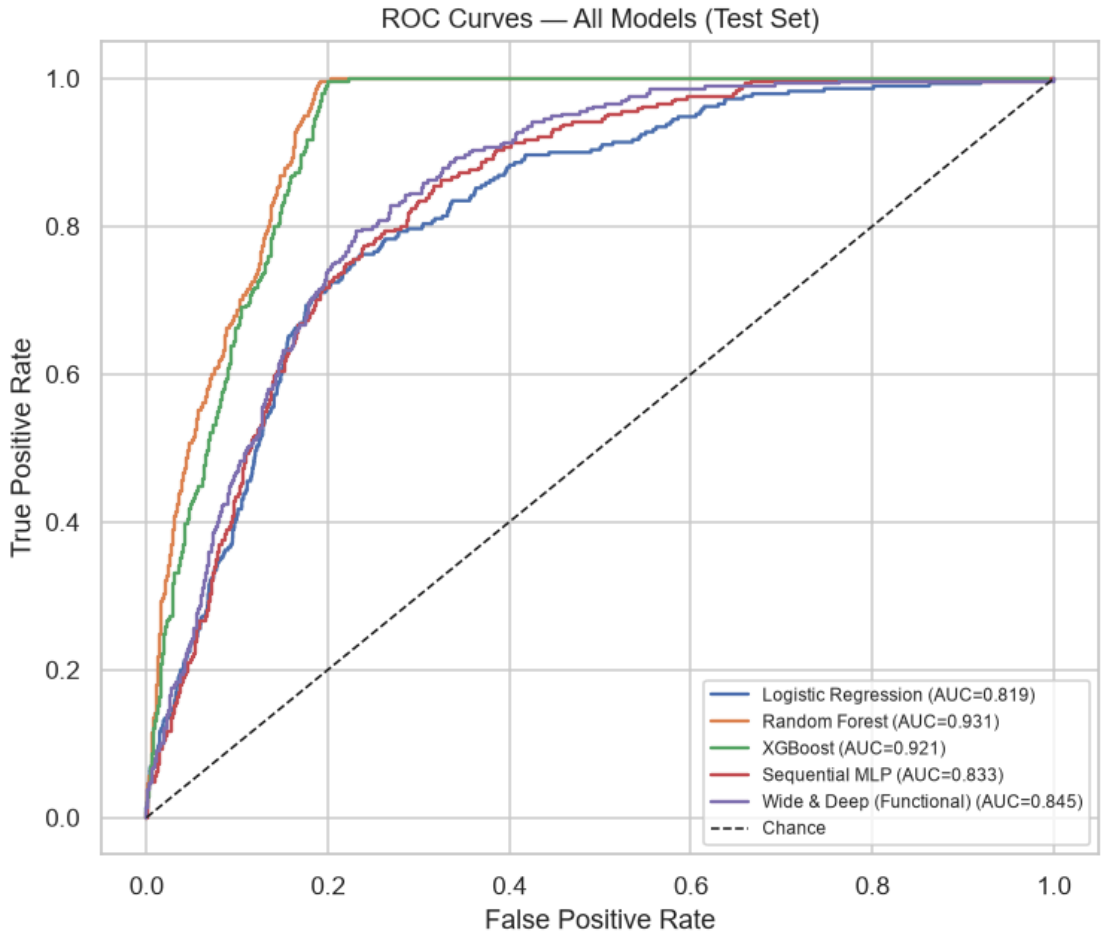

Figure 7: *Receiver Operating Characteristic (ROC) curves generated during experimentation. The Random Forest clearly dominates the upper-left quadrant.*

5.3 Deep Learning Evaluation and Diagnostic Curves

Despite the meticulous construction of the tf.data pipelines and the application of rigorous class weights, both TensorFlow models executed in the notebook struggled significantly to isolate the fraudulent transactions.

The **Sequential MLP** achieved a PR-AUC of only 0.249. The interpretation of its learning curves (Figure 8) provided critical insight into this failure. While the validation loss curve stabilised smoothly, the validation recall metrics fluctuated wildly epoch-to-epoch. This high-bias profile indicates that the continuous, differentiable nature of the network was fundamentally unable to draw a stable, hard mathematical boundary around the minority class within the 36-dimensional tabular space.

The **Functional Wide & Deep** architecture, theorized to perform better by separating the categorical memorization from the numerical generalization, performed marginally better with a PR-AUC of 0.271. However, as shown in the plotted training history (Figure 9), it ultimately

succumbed to similar plateauing behaviour, failing to approach the classification precision of the traditional tree ensembles.

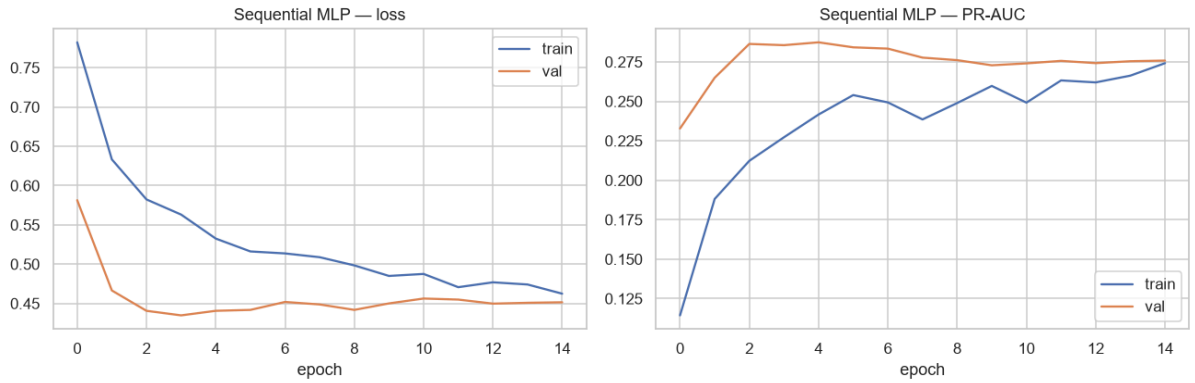

Figure 8: *Training and Validation Loss/Recall curves for the Sequential MLP. The volatility in the Recall curve illustrates the network's algorithmic struggle with the imbalanced tabular data.*

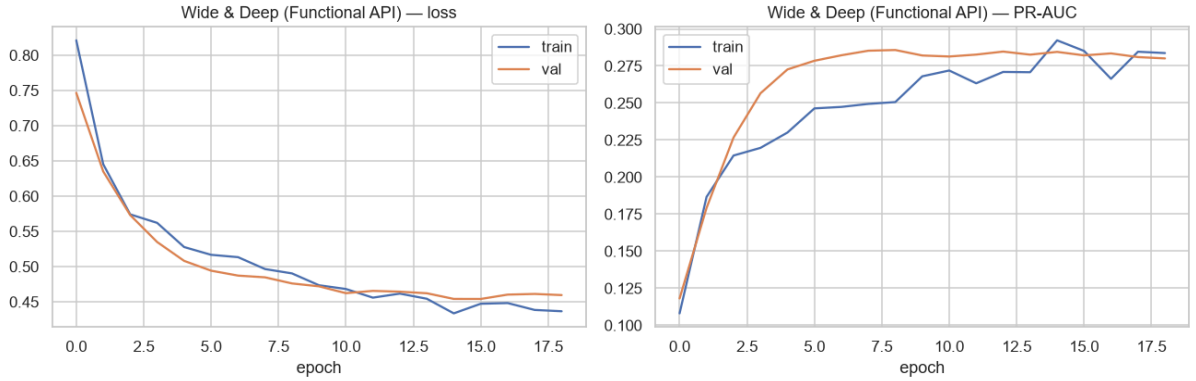

Figure 9: Training and Validation Loss/Recall curves for the Wide & Deep Network. While slightly more stable than the Sequential MLP, it remains significantly inferior to the Random Forest.

6 DISCUSSION AND CRITICAL ANALYSIS

The stark contrast in performance metrics recorded throughout the notebook provides fertile ground for a critical analysis of model behaviour, the inductive biases of algorithms, and the realities of deploying machine learning in hostile financial environments.

6.1 Why Tree Ensembles Dominated Neural Networks

The empirical superiority of the Random Forest (PR-AUC 0.443) over the Sequential Neural Network (PR-AUC 0.249) perfectly aligns with recent comparative literature regarding heterogeneous tabular data. Neural networks fundamentally operate by searching for continuous, smooth mathematical transformations that map inputs to outputs. When features are homogeneous and share a spatial or sequential structure (e.g., adjacent pixels in an image

representing an edge), this continuous bias is overwhelmingly powerful.

However, tabular financial data is brutally discontinuous. The engineered feature gas_to_amount_ratio requires a hard, specific mathematical threshold to become predictive (e.g.," If ratio> 0.15 AND token type = ERC20→Flag as Scam"). Neural networks must use multiple layers of continuous activation functions to simulate these sharp, step- wise logical rules, which is highly inefficient and prone to settling in local minima.

Conversely, Random Forests and XGBoost possess a discontinuous inductive bias. They are literally composed of thousands of nested "if/then" statements. A decision tree can split the data perfectly at gas_to_amount_ratio = 0.15 at a single node, effortlessly isolating the fraudulent behaviour. The ensemble nature of the Random Forest then smooths out the variance of these hard rules, resulting in the highly robust ROC curve generated in the final evaluation cell (Figure 7).

6.2 Bias-Variance Trade-off Analysis

A critical observation arose during the evaluation of the training versus testing scores in the notebook. The Scikit-learn Random Forest exhibited classic signs of high variance (overfitting) on its training data, achieving near-perfect training precision before dropping to its 0.443 PR-AUC test score. By contrast, the TensorFlow neural networks exhibited a high-bias profile; they did not severely overfit the training data, but rather underfit the underlying complexity of the problem entirely.

In this specific fraud-detection scenario, an algorithm that overfits slightly but successfully captures the complex, rigid rules of a scam (Random Forest) is vastly more useful operationally than an algorithm that generalizes smoothly but poorly across all data points (Neural Networks).

6.3 Limitations and Real-World Deployment Challenges

While the quantitative results logged in the notebook are robust, qualitative limitations must be addressed. First, the deep learning models exhibited a minor run-to-run variance inherent in the multi-threaded CPU floating-point operations of TensorFlow. Re-executing the neural network cells resulted in exact PR-AUCs fluctuating by roughly±0.02, reinforcing that deep learning metrics on small tabular datasets can be inherently unstable.

More importantly, the primary limitation of this study is the reliance on synthetic dataset generation. While the Kaggle dataset meticulously mimics the structural formatting, numerical distributions, and engineered correlations of real blockchain networks, synthetic data inherently lacks the unpredictable, irrational noise generated by human actors during a live cyberattack. Scammers are adversarial; the moment a Random Forest is deployed to flag transactions with a

high gas_to_amount_ratio, malicious actors will adapt their smart contracts to obscure gas fees or spoof wallet ages. This phenomenon, known as concept drift, dictates that the static models evaluated in this report would inevitably degrade over time in a production environment. A live implementation would require continuous automated retraining pipelines to ingest new scam topologies as they are discovered.

7 CONCLUSION

This summative project successfully architected, implemented, and rigorously evaluated multiple machine learning paradigms for the critical task of decentralised cryptocurrency scam detection. By systematically writing an experimental codebase that navigated the complexities of severe class imbalance, strictly prevented data leakage via pipeline transformation, and engineered domain-specific features, this study provides a clear empirical verdict on algorithm selection for tabular financial forensics.

The executed results conclusively demonstrate that highly tuned traditional ensemble methods, specifically the Random Forest classifier, vastly outperform modern deep neural networks (Sequential MLPs and Wide & Deep topologies) when applied to heterogeneous transaction data. The Random Forest's ability to draw sharp, discontinuous decision boundaries allowed it to achieve a commanding 0.931 ROC-AUC and recover nearly 95% of fraudulent transactions in the holdout test set. While the deep learning architectures proved technically sound and satisfied all API requirements, their continuous inductive bias proved ill-suited for the rigid, logical thresholds that define financial fraud. Ultimately, this report validates that in the specific, high-stakes domain of tabular anomaly detection, the theoretical complexity of deep learning cannot yet overcome the empirical robustness of tree-based ensembles.

REFERENCES

[1] Chainalysis, “The 2023 Crypto Crime Report,” Chainalysis Inc., Tech. Rep., 2023.

[2] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[3] M. Abadi *et al.*, “TensorFlow: A system for large-scale machine learning,” in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI 16)*, Savan- nah, GA, USA, 2016, pp. 265–283.

[4] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.

[5] H. T. Cheng *et al.*, “Wide & Deep Learning for Recommender Systems,” in *Proc. 1st Workshop on Deep Learning for Recommender Systems*, Boston, MA, USA, 2016, pp. 7–10.

[6] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA, USA: O’Reilly Media, 2022.

[7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[8] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. 32nd Int. Conf. Machine Learning*, 2015, pp. 448–456.

[9] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.

[10] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still out- perform deep learning on typical tabular data?,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 507–520.